🏢 Edulateral Foundation — Official Evaluative Assessment Bank

Day 4: Distributed AI Training, Slurm Cluster Workflows & Intel VTune Profiler

Target Audience: GLA University 7th Semester B.Tech Computer Science & Engineering
Lead Trainer: Shivam Gupta | **Evaluation Mode**: Objective Technical MCQ Assessment
Total Questions: 25 Questions | **Time Limit**: 45 Minutes

Question 01: D4-Q01

What is the Top-Down Microarchitecture Analysis Method (TMAM) implemented in Intel VTune Profiler?

- **(A)** A method to organize project source code directories.
- **(B)** A hierarchical diagnostic methodology based on CPU Performance Monitoring Unit (PMU) counters that categorizes processor execution pipelines into Retiring, Bad Speculation, Front-End Bound, and Back-End Bound.
- **(C)** A top-down management style for software companies.
- **(D)** A network routing protocol for Wi-Fi routers.

> **Correct Answer**: **(B)**
> **Technical Explanation**: TMAM (developed by Ahmad Yasin at Intel) classifies CPU execution pipeline slot bottlenecks into four primary top-level categories: Front-End Bound, Bad Speculation, Back-End Bound, and Retiring. This allows performance engineers to identify exactly why CPU cycles are stalled.

Question 02: D4-Q02

In Intel VTune Profiler TMAM analysis, what does a high 'Back-End Bound' metric (>40%) indicate?

- **(A)** The CPU is waiting for instructions to be fetched and decoded.
- **(B)** The CPU pipeline has successfully retired all instructions.
- **(C)** The processor is stalled waiting for data memory access (Memory Bound: L1/L2/L3 cache misses, DRAM latency) or execution port contention (Core Bound).
- **(D)** The network cable is disconnected.

> **Correct Answer**: **(C)**
> **Technical Explanation**: Back-End Bound indicates that execution units are stalled waiting for data from the memory hierarchy (Memory Bound) or because execution units/divider pipelines are saturated (Core Bound).

Question 03: D4-Q03

In VTune, what does a high 'Bad Speculation' metric indicate?

- **(A)** The engineer made a poor financial investment in stock markets.

- **(B)** CPU execution slots were wasted computing instructions along branch paths that were incorrectly predicted by branch predictors and subsequently flushed.
- **(C)** The hard drive has corrupted sectors.
- **(D)** The power supply voltage is fluctuating.

> **Correct Answer**: **(B)**
> **Technical Explanation**: Bad Speculation measures pipeline slots wasted due to branch mispredictions. When a conditional branch is mispredicted, all speculative work in the out-of-order execution pipeline must be cancelled and flushed, wasting CPU cycles and energy.

Question 04: D4-Q04

What CPU hardware mechanism is sampled by Intel VTune Profiler to measure hardware events without modifying application source code?

- **(A)** Performance Monitoring Unit (PMU) hardware event counters embedded in silicon cores.
- **(B)** Motherboard audio microphone.
- **(C)** GPU HDMI display output.
- **(D)** Power supply fan tachometer.

> **Correct Answer**: **(A)**
> **Technical Explanation**: Modern Intel processors include dedicated hardware Performance Monitoring Unit (PMU) registers that count low-level microarchitectural events (CPU_CLK_UNHALTED, INST_RETIRED, MEM_LOAD_RETIRED.L3_MISS) with near-zero overhead.

Question 05: D4-Q05

What is the command-line syntax to run an Intel VTune Hotspots analysis on a Python distributed training script?

- **(A)** vtune -collect hotspots -result-dir ./vtune_out -- python train.py
- **(B)** vtune --delete train.py
- **(C)** python vtune train.py
- **(D)** perf format ./vtune_out

> **Correct Answer**: **(A)**
> **Technical Explanation**: `vtune -collect hotspots -result-dir ./vtune_out -- <command>` launches the VTune sampling collector to profile function execution times, call stacks, and CPU hotspots, outputting the results into the specified directory.

Question 06: D4-Q06

In distributed HPC scaling analysis, how is Parallel Scaling Efficiency (E) calculated when scaling from 1 node to N nodes?

- **(A)** $E = \frac{Speedup(N)}{N} = \frac{T_1}{N \times T_N} \times 100\%$
- **(B)** $E = N \times T_1$
- **(C)** $E = T_N - T_1$

- **(D)** $E = \frac{N}{T_N}$

> **Correct Answer**: **(A)**
> **Technical Explanation**: Scaling efficiency measures how effectively an application exploits additional compute nodes. Ideal linear scaling yields $E = 100\%$. If 4 nodes take 30 seconds compared to 100 seconds on 1 node, $Speedup = 100/30 = 3.33$, so $E = 3.33/4 = 83.3\%$.

Question 07: D4-Q07

What is the primary cause of parallel scaling efficiency degradation as an HPC cluster scales from 8 nodes to 128 nodes in distributed deep learning?

- **(A)** Communication overhead (AllReduce latency, network synchronization, and barrier wait times) beginning to exceed local compute time.
- **(B)** The CPU runs out of memory registers.
- **(C)** Python interpreter files wear out.
- **(D)** InfiniBand cables become warm.

> **Correct Answer**: **(A)**
> **Technical Explanation**: As worker count scales, local computation per node decreases (in strong scaling), while inter-node communication volume and synchronization latency (network jitter, ring latency, stragglers) increase, degrading overall efficiency.

Question 08: D4-Q08

What is Intel Trace Analyzer and Collector (ITAC) used for in an HPC cluster environment?

- **(A)** Visualizing and profiling MPI communication patterns, identifying load imbalances, message serialization, and MPI wait bottlenecks across cluster nodes.
- **(B)** Tracking student attendance in computer labs.
- **(C)** Monitoring room humidity and temperature.
- **(D)** Scanning for malware.

> **Correct Answer**: **(A)**
> **Technical Explanation**: ITAC instruments MPI applications to generate trace files displaying exact timelines of MPI function calls (`MPI_Waitall` , `MPI_Bcast` , `MPI_Allreduce`), helping engineers visualize communication imbalances across cluster nodes.

Question 09: D4-Q09

In VTune Memory Access Analysis, what metric flags that an application is stalling due to accessing DRAM on a remote socket across UPI links?

- **(A)** Remote DRAM Access / Cross-Socket Traffic percentage
- **(B)** CPU Fan RPM
- **(C)** L1 Cache Hit Rate = 100%
- **(D)** DRAM Power Off

> **Correct Answer**: **(A)**
> **Technical Explanation**: VTune's Memory Access analysis queries PMU counters tracking remote vs local NUMA node DRAM loads (`OCR.DEMAND_DATA_RD.REMOTE_DRAM`), flagging applications suffering from improper thread pinning or memory affinity.

Question 10: D4-Q10

What is a 'Flame Graph' in performance profiling and VTune analysis?

- **(A)** A thermal heatmap of computer CPU temperature.
- **(B)** A visual representation of hierarchical call stacks where the x-axis represents the proportion of CPU time spent in each function and the y-axis represents stack depth.
- **(C)** A fire warning indicator in server rooms.
- **(D)** A graph of electrical voltage spikes.

> **Correct Answer**: **(B)**
> **Technical Explanation**: Flame Graphs (invented by Brendan Gregg) visualize profiled call stacks. The width of each rectangular box is proportional to the total time spent executing that function (and its children), making bottlenecks instantly recognizable.

Question 11: D4-Q11

What command-line tool is used to generate a text summary report of an existing VTune analysis result without opening the graphical GUI?

- **(A)** vtune -report summary -result-dir ./vtune_out
- **(B)** cat ./vtune_out/report.txt
- **(C)** echo vtune
- **(D)** vtune --format-disk

> **Correct Answer**: **(A)**
> **Technical Explanation**: `vtune -report <report_type> -result-dir <dir>` parses raw collected database files and prints formatted text tables of top hotspot functions, CPU utilization, and microarchitecture breakdown directly to the terminal.

Question 12: D4-Q12

In VTune Threading Analysis, what does high 'Lock Contention' indicate in a multi-threaded C++/Python application?

- **(A)** Worker threads spend excessive time blocked waiting for shared mutexes or locks to be released by other threads, serializing parallel execution.
- **(B)** The computer chassis padlock is locked.
- **(C)** The user is locked out of their Linux account.
- **(D)** The CPU frequency is locked to maximum.

> **Correct Answer**: **(A)**
> **Technical Explanation**: High Lock Contention indicates that threads are serialized at synchronization points (e.g., pthread mutexes, OpenMP critical sections, or Python's GIL), causing cores to waste cycles waiting instead of performing work.

Question 13: D4-Q13

What is the Roofline Model used for in high-performance computing performance engineering?

- **(A)** Measuring the structural roof integrity of a data center building.
- **(B)** A 2D visual performance model relating an application's Operational Intensity (FLOPs/Byte) to hardware peak compute performance and peak memory bandwidth, revealing whether an algorithm is compute-bound or memory-bound.
- **(C)** Calculating solar panel electrical output.
- **(D)** Estimating network switch rack height.

> **Correct Answer**: **(B)**
> **Technical Explanation**: The Roofline Model plots computational performance (GFLOP/s) against operational intensity (FLOPs per byte of DRAM traffic). The horizontal ceiling represents peak compute throughput, while the sloped boundary represents peak memory bandwidth, showing developers the exact performance headroom available.

Question 14: D4-Q14

If an application's operational intensity falls to the left of the Roofline knee point, what type of optimization will yield the greatest performance improvement?

- **(A)** Adding more floating-point compute units.
- **(B)** Memory access optimizations (cache blocking, loop tiling, data layout transformation from Array of Structures to Structure of Arrays) to reduce DRAM memory bandwidth pressure.
- **(C)** Increasing CPU clock frequency only.
- **(D)** Rewriting the code in Python.

> **Correct Answer**: **(B)**
> **Technical Explanation**: Workloads to the left of the Roofline knee are memory-bandwidth bound. Adding more compute units has zero effect because execution units are starved for data. Optimizing cache reuse and data layout directly moves the application closer to peak hardware throughput.

Question 15: D4-Q15

What is the impact of SIMD vector register spilling to the stack on CPU execution performance?

- **(A)** It speeds up matrix multiplications by 10x.
- **(B)** It severely degrades performance because vector values that should reside in fast ZMM registers are repeatedly saved and reloaded from L1 cache/memory, increasing instruction count and latency.
- **(C)** It reduces memory usage to zero.
- **(D)** It disables hyperthreading.

> **Correct Answer**: **(B)**
> **Technical Explanation**: Register spilling occurs when an inner loop requires more vector registers than the 32 physical ZMM registers available in AVX-512. The compiler must emit stack push/pop instructions, saturating execution ports with load/store operations.

Question 16: D4-Q16

What is the primary benefit of Loop Tiling (Cache Blocking) in high-performance numerical matrix multiplication?

- **(A)** It partitions large matrices into smaller sub-blocks that fit entirely within L1 or L2 CPU cache memory, enabling massive data reuse and eliminating slow DRAM accesses.
- **(B)** It displays graphics tiles on computer screens.
- **(C)** It removes loops from the code entirely.
- **(D)** It saves disk space.

> **Correct Answer**: **(A)**
> **Technical Explanation**: For large matrices ($N > 2048$), naive triple loops access memory with stride N, causing L1/L2 cache evictions on every iteration. Loop tiling divides matrices into $B \times B$ blocks sized to fit inside cache, ensuring each data element is reused multiple times before eviction.

Question 17: D4-Q17

In distributed Slurm training, what is the role of the environment variable `SLURM_PROCID`?

- **(A)** It represents the global MPI rank of the current task process across all nodes in the job allocation.
- **(B)** It is the student's admission roll number.
- **(C)** It is the password for the database.
- **(D)** It represents the processor voltage.

> **Correct Answer**: **(A)**
> **Technical Explanation**: `SLURM_PROCID` is exported by Slurm to each spawned task process, indicating its global rank (0 to $N-1$). Distributed frameworks read this variable to configure MPI rank assignments and communication topologies.

Question 18: D4-Q18

What is 'Straggler Effect' in synchronous distributed deep learning across a 64-node cluster?

- **(A)** A single slow or throttled compute node delays the entire cluster at the AllReduce synchronization barrier, reducing overall training throughput to match the slowest node.
- **(B)** Nodes running out of disk space.
- **(C)** Nodes receiving conflicting IP addresses.
- **(D)** A virus spreading across Ethernet cables.

> **Correct Answer**: **(A)**
> **Technical Explanation**: Synchronous Ring-AllReduce requires all nodes to participate in every barrier synchronization step. If one node experiences thermal throttling, OS background interrupts, or bad memory channels, all other 63 nodes sit completely idle waiting for it to arrive at the barrier.

Question 19: D4-Q19

What VTune analysis type is specifically designed to identify SIMD vectorization efficiency and whether loops are scalar or vectorized with AVX-512?

- **(A)** HPC Performance Characterization / Vectorization Analysis
- **(B)** Audio Waveform Analysis
- **(C)** Disk Defragmenter
- **(D)** Web Browser Profiler

> **Correct Answer**: **(A)**
> **Technical Explanation**: VTune's HPC Performance Characterization analysis measures SIMD Vectorization Intensity, calculating what percentage of floating-point operations were vectorized with AVX-512 vs AVX2 vs scalar instructions.

Question 20: D4-Q20

What does a high Instruction-Per-Cycle (IPC) metric (e.g. IPC > 2.5) typically indicate in CPU execution analysis?

- **(A)** The processor pipeline is executing instructions efficiently with minimal memory or branch stalls.
- **(B)** The CPU is overheating.
- **(C)** The application has crashed.
- **(D)** The network connection is slow.

> **Correct Answer**: **(A)**
> **Technical Explanation**: IPC measures superscalar throughput. High IPC (>2.0 on modern Intel architectures) indicates that out-of-order execution units are retiring multiple instructions per clock cycle with excellent cache hit rates and predictable branch patterns.

Question 21: D4-Q21

In Slurm, what does the command `sbatch --dependency=afterok:12345 pipeline.slurm` accomplish?

- **(A)** It submits `pipeline.slurm` to run only after job 12345 completes successfully with exit code 0.
- **(B)** It cancels job 12345.
- **(C)** It runs both jobs simultaneously on the same core.
- **(D)** It deletes job 12345's output files.

> **Correct Answer**: **(A)**
> **Technical Explanation**: Slurm job dependencies allow chaining complex multi-stage workflows (e.g. data preprocessing -> distributed training -> model profiling -> evaluation). `--dependency=afterok:<job_id>` holds the downstream job until the prerequisite completes cleanly.

Question 22: D4-Q22

What is the primary difference between Strong Scaling and Weak Scaling benchmarks in high-performance computing?

- **(A)** Strong scaling keeps total problem size constant while increasing core count; Weak scaling increases problem size proportionally to core count, keeping work per core constant.
- **(B)** Strong scaling uses C++; weak scaling uses Python.
- **(C)** Strong scaling is for hardware; weak scaling is for software.
- **(D)** There is no difference.

> **Correct Answer**: **(A)**
> **Technical Explanation**: Strong scaling tests how quickly a fixed problem can be solved by adding processors (governed by Amdahl's Law). Weak scaling tests how large a problem can be solved in the same amount of time as compute power expands (governed by Gustafson's Law).

Question 23: D4-Q23

What does Intel Inspector detect in parallel C/C++ HPC codes?

- **(A)** Memory leaks, uninitialized memory accesses, buffer overflows, and multi-threaded data races.
- **(B)** Grammatical errors in code comments.
- **(C)** The author's identity.
- **(D)** Internet connection speed.

> **Correct Answer**: **(A)**
> **Technical Explanation**: Intel Inspector is a memory and threading error debugger. It dynamically instruments compiled binaries to detect data races, deadlocks, illegal memory accesses, and memory leaks without requiring special compiler flags.

Question 24: D4-Q24

Why is Structure of Arrays (SoA) layout generally superior to Array of Structures (AoS) for SIMD vectorization in HPC applications?

- **(A)** SoA stores contiguous values of identical fields adjacent in memory, allowing SIMD vector units to execute aligned unit-stride loads without expensive gather/scatter instructions.
- **(B)** SoA reduces file size on disk.
- **(C)** AoS requires more RAM.
- **(D)** SoA is supported only by Python.

> **Correct Answer**: **(A)**
> **Technical Explanation**: In AoS (`struct { float x, y, z; } points[N]`), reading all `x` values requires strided memory loads with stride 3, which vector units cannot load in a single contiguous read. In SoA (`struct { float x[N], y[N], z[N]; }`), elements of `x` reside in contiguous memory, allowing single-instruction aligned 512-bit vector loads (`_mm512_load_ps`).

Question 25: D4-Q25

What Linux tool allows an engineer to inspect real-time PCIe bandwidth utilization and link generation speeds (e.g. PCIe Gen 5 x16) on an HPC accelerator node?

- **(A)** lspci -vvv
- **(B)** ping localhost
- **(C)** free -m
- **(D)** uname -a

> **Correct Answer**: **(A)**
> **Technical Explanation**: `lspci -vvv` queries the PCI configuration space, displaying link capabilities and status (e.g. `LnkCap: Port #0, Speed 32GT/s, Width x16` and `LnkSta: Speed 32GT/s (PCIe Gen 5), Width x16`), verifying that accelerators operate at full bus bandwidth.
